5b. Mutable and Immutable Objects

Martin Alfaro
PhD in Economics

INTRODUCTION

Objects in programming can be broadly classified into two categories: mutable and immutable. **Mutable objects** permit modification of their internal state after creation. This means that their elements can be modified, appended, or removed at will, thus providing a high degree of flexibility. A prime example is vectors.

In contrast, **immutable objects** can't be altered after their creation: they prevent additions, removals, or modifications of their elements. A common example of immutable object in Julia is **tuples**. Immutability effectively locks variables into a read-only state, safeguarding against unintended changes. Simultaneously, it can result in performance gains for small objects, as we'll show in Part II of this book.

This section will be relatively brief, focusing solely on the distinctions between mutable and immutable objects. Subsequent sections will expand on their uses and properties.

> **Remark**
>
> A popular package called `StaticArrays` provides an implementation of **immutable vectors**. We'll explore them in the context of high performance, as they can greatly speed up computations involving small vectors. They're created using tuples under the hood.

EXAMPLES OF MUTABILITY AND IMMUTABILITY

To illustrate, the following examples attempt to modify existing elements of a collection. The examples rely on vectors as an example of a mutable object and tuples for immutable ones. Additionally, we present the case of strings as another example of immutable object, which are essentially sequences of characters.

MUTABILITY OF VECTORS

```
x = [3,4,5]
```

```
julia> x[1] = 0

julia> x
3-element Vector{Int64}:
 0
 4
 5
```

IMMUTABILITY OF TUPLES

```
x = (3,4,5)
```

```
julia> x[1] = 0
ERROR: MethodError: no method matching setindex!(::Tuple{Int64, Int64, Int64}, ::Int64,
::Int64)
```

IMMUTABILITY OF STRINGS

```
x = "hello"
```

```
julia> x
'h': ASCII/Unicode U+0068 (category Ll: Letter, lowercase)

julia> x[1] = 'a'
ERROR: MethodError: no method matching setindex!(::String, ::Char, ::Int64)
```

The key characteristic of mutable objects is their ability to modify existing elements. Additionally, it commonly allows for the dynamic addition and removal of elements. In a subsequent section, we'll present various methods for implementing this last functionality. For now, we simply demonstrate the concept by using the functions `push!` and `pop!`, which respectively add and remove an element at the end of a collection.

ADD (MUTABLE OBJECT)

```
x = [3,4]

push!(x, 5)       # add element 5 at the end
```

```
julia> x
3-element Vector{Int64}:
 3
 4
 5
```

REMOVE (MUTABLE OBJECT)

```
x = [3,4,5]

pop!(x)           # delete last element
```

```
julia> x
2-element Vector{Int64}:
 3
 4
```

ADD/REMOVE (IMMUTABLE OBJECT)

```
x = (3,4,5)

pop!(x)           # ERROR
push!(x,6)        # ERROR
```

```
ERROR: MethodError: no method matching pop!(::Tuple{Int64, Int64, Int64})
```